IMPERIAL

Machine Learning for Design Engineers

Week 2: Neurons and Artificial Neural Networks

Pietro Ferraro

Table of Content

The Neuron

Background

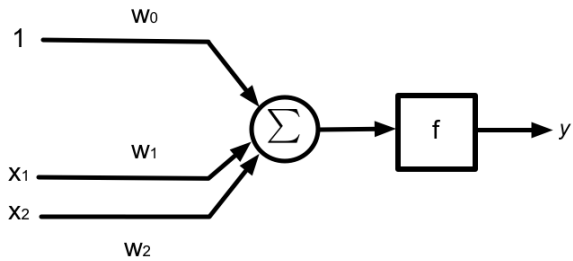
The term Artificial Neural Network means any architecture that has massively parallel interconnections of simple elements.

Sometimes such architectures are motivated by similarities to real neural networks. We won't concern ourselves with such justifications, and just note that such architectures are extremely useful learning to solve both regression and classification tasks.

The best known such network is the Multi-layer Perceptron (MLP).

The neuron

The building block for Deep Learning is the neuron

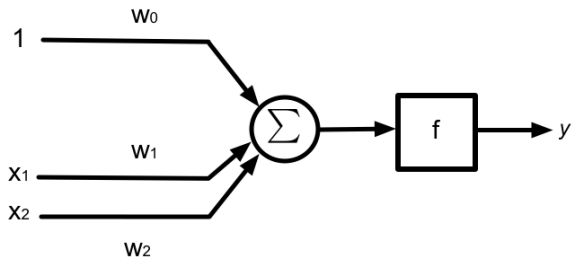


$$y = f(\sum_{i=1}^{D} w_i x_i + w_0) = f(w^T x + w_0)$$

Where $w = [w_1, w_2, ..., w_D]^T$ and $x = [x_1, x_2, ..., x_N]^T$ and $f(\cdot)$ is a non-linear function.

The neuron

The building block for Deep Learning is the neuron

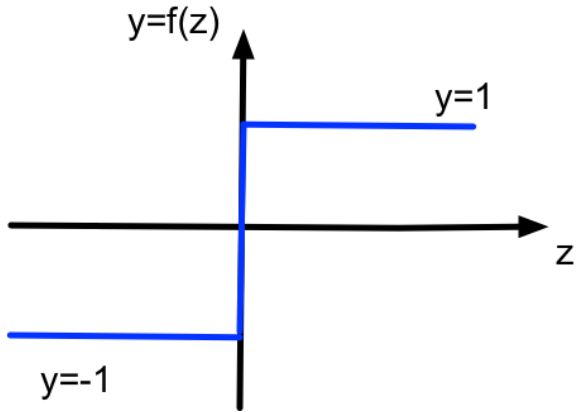


$$y = f(\sum_{i=1}^{D} w_i x_i + w_0) = f(w^T x + w_0)$$

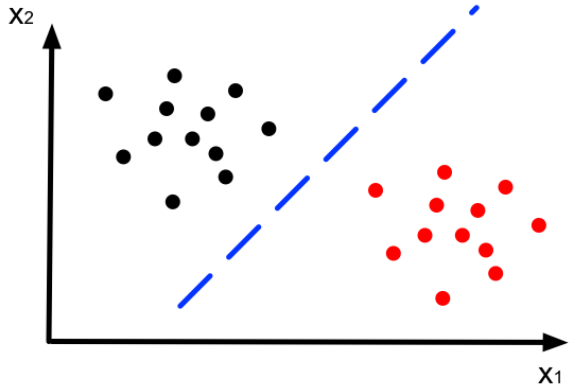
Why this structure? And why should $f(\cdot)$ be non linear?

Linear separation

Consider one specification of $f(\cdot)$



Linear separation



Linear separation

Data-sets in which class membership can be determined using a line, or more generally, a hyperplane, are said to be linearly separable. In this case, the learning task is to find a separating hyperplane of the form $w_1x_1 + w_2x_2 + w_0 = 0$ with the property that for an appropriate function $f(\cdot)$

$$f(w_1x_1 + w_2x_2 + w_0) = y \tag{1}$$

where $y = 1$ for all $w_1x_1 + w_2x_2 + w_0 > 0$ and $y = -1$ otherwise (the labels for each class is -1 and 1 respectively).

Linear separation

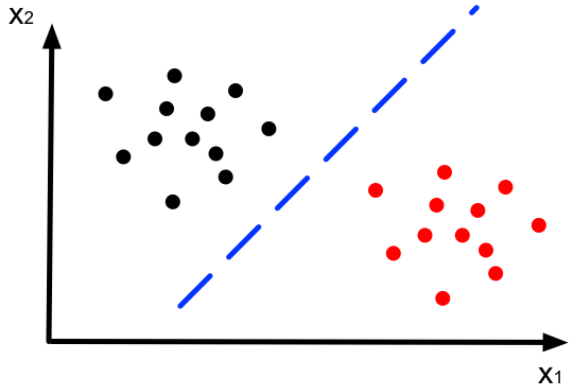
Linearly seperable:

$$f(w_1 x_1 + w_2 x_2 + w_0) = y \tag{2}$$

where $y = 1$ for all $w_1 x_1 + w_2 x_2 + w_0 > 0$ and $y = -1$ otherwise (the labels for each class is -1 and 1 respectively).

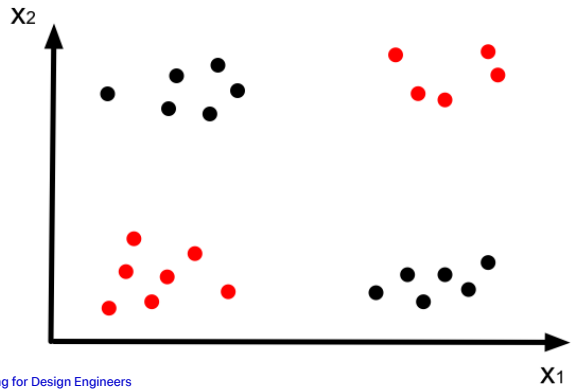
Recall that a hyperplane splits the plane into two parts: the set of points $x = (x_1, x_2)$ satisfying $w_1 x_1 + w_2 x_2 + w_0 > 0$ and $w_1 x_1 + w_2 x_2 + w_0 < 0$.

Linear separation



Linear separation

Now consider a more complicated pattern recognition problem. Lets not concern ourselves with algorithms to find the parameters of the network; rather lets think about what types of problems this network can solve?



Linear separation

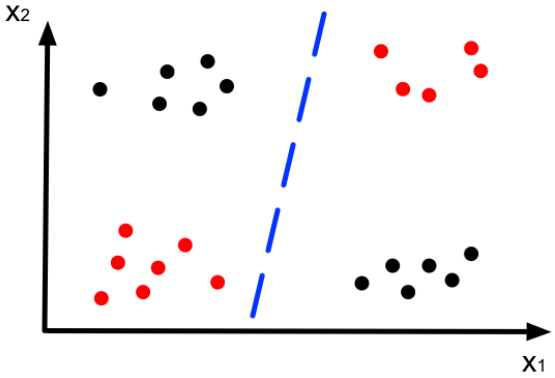


Figure: Cannot separate the classes

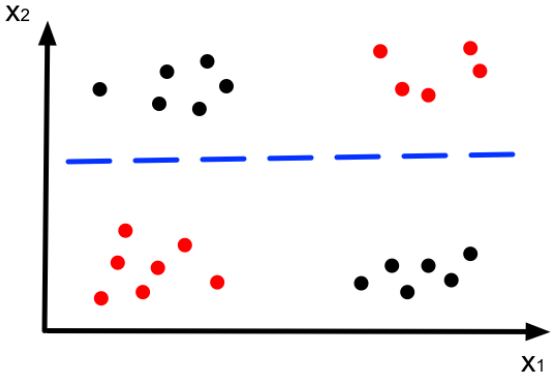


Figure: Cannot separate the classes

Linear separation

Now let us consider a slightly more complicated architecture.

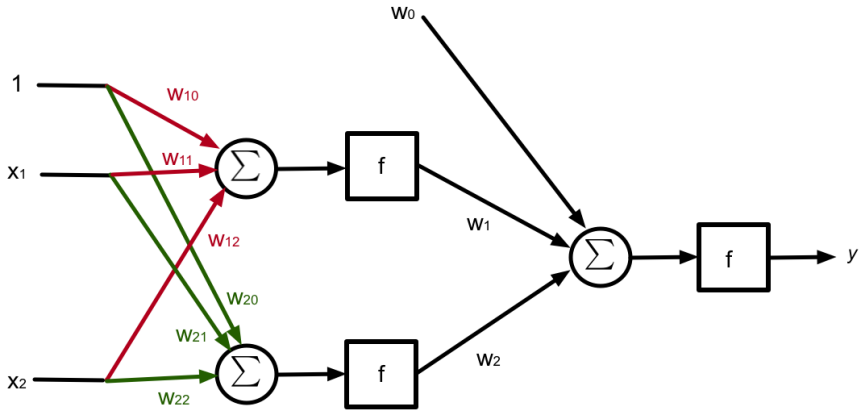


Figure: A multi-layer architecture with two inputs

Linear separation

The first layer of the architecture has two inputs and two outputs.

$$f(w_{11}x_1 + w_{12}x_2 + w_{10}) = y_1 \quad (3)$$
$$f(w_{21}x_1 + w_{22}x_2 + w_{20}) = y_2 \quad (4)$$

In the second layer there are also two inputs and the output is

$$f(w_1y_1 + w_2y_2 + w_0) = y \quad (5)$$

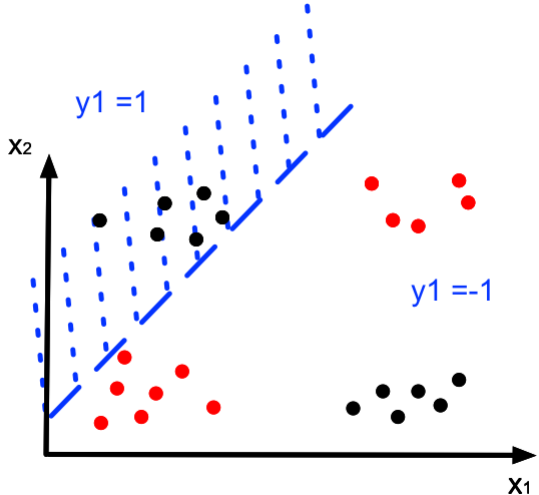
Linear separation

We can immediately see that this is a much richer architecture.

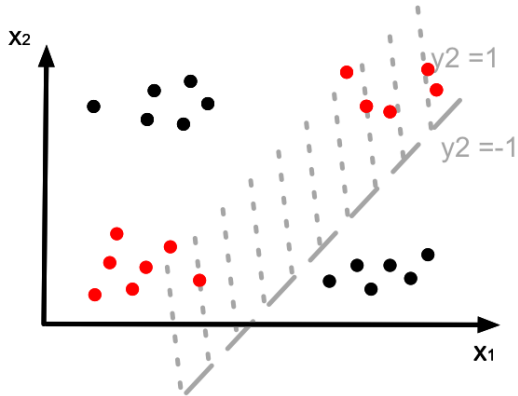
The first layer constructs two classifiers and separates classes linearly as before.

The second layer the constructs a hyperplane to classify the output of these classifiers.

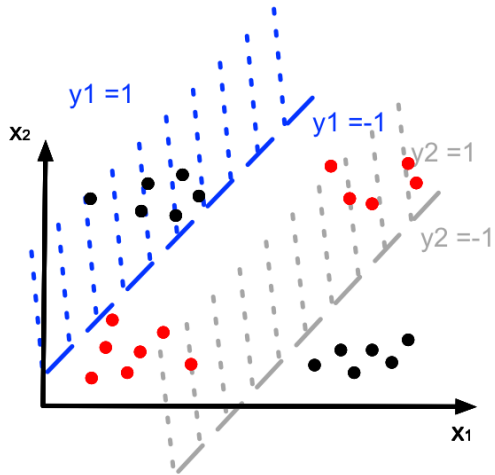
Linear separation



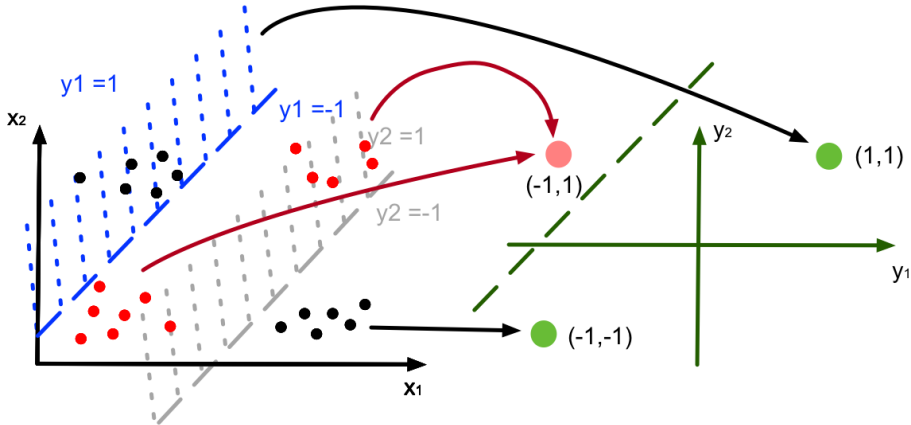
Linear separation



Linear separation



Linear separation



Linear separation

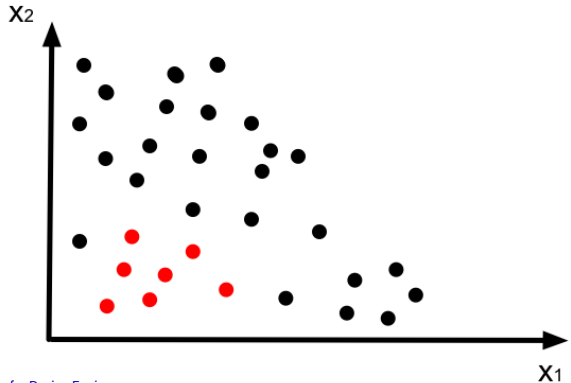
By moving to a higher dimensional space we make linear discrimination possible.

In fact it is known that pattens classified as using nonlinear functions, can be separated linearly in a higher dimensional space.

The price of doing this is that learning becomes more difficult.

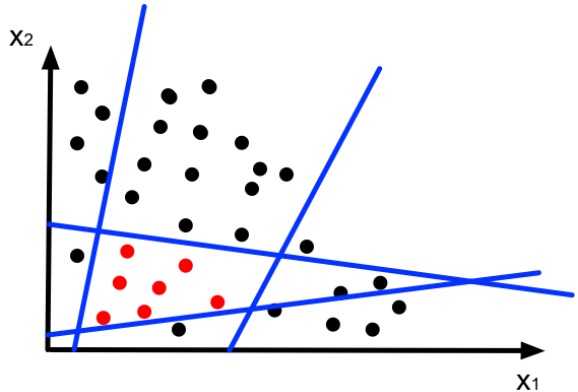
More general pattern recognition problems

We can already see the power of this architecture. Now consider a more complicated pattern recognition task.



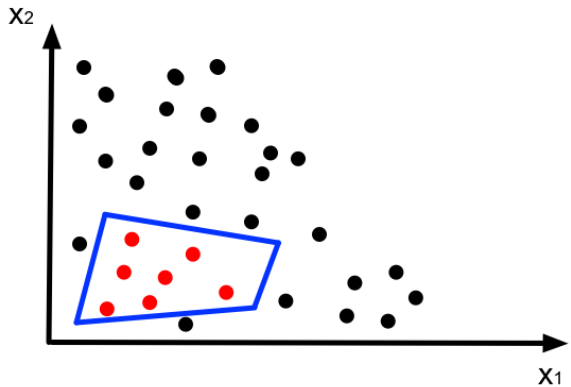
More general pattern recognition problems

We can use linear discriminators to construct a decision boundary.



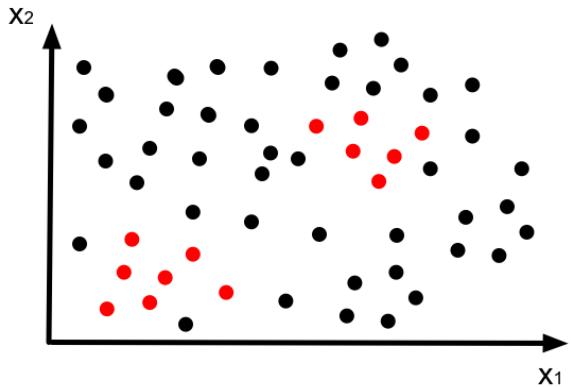
More general pattern recognition problems

We can use linear discriminators to construct a decision boundary.



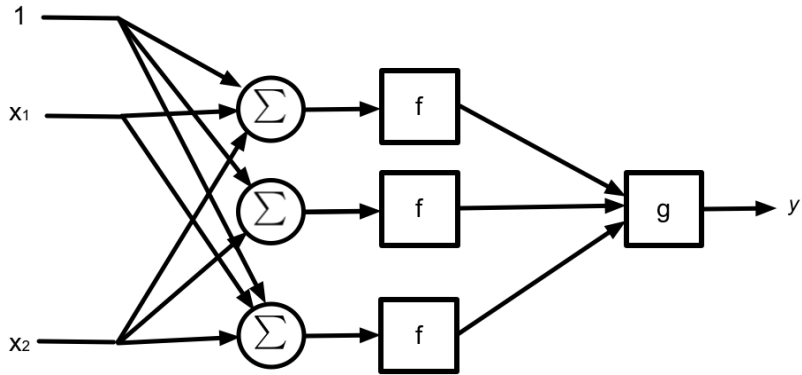
More general pattern recognition problems

We can even handle really complicated situations.



More general pattern recognition problems

And this can be realised using a multi-layer architecture.



Artificial Neural networks

The general version of this architecture is an Artificial Neural Network (ANN).

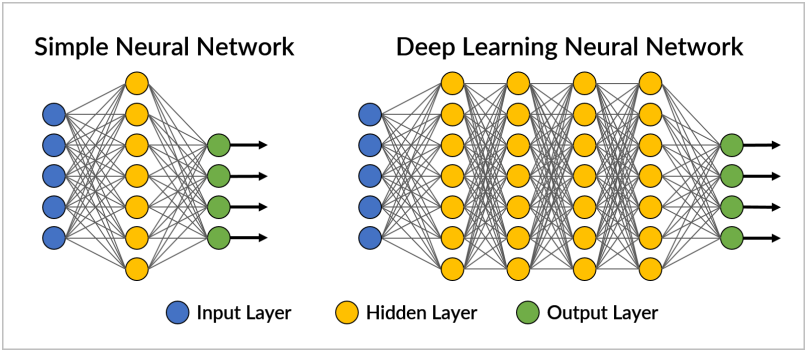
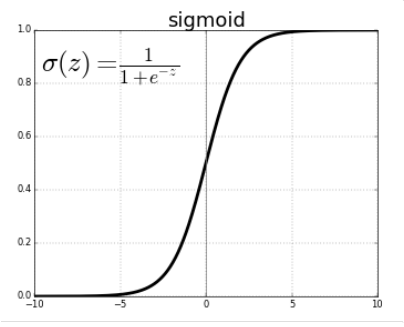


Figure: Source

Activation Functions

Usual choices for $f(\cdot)$ are as follows:



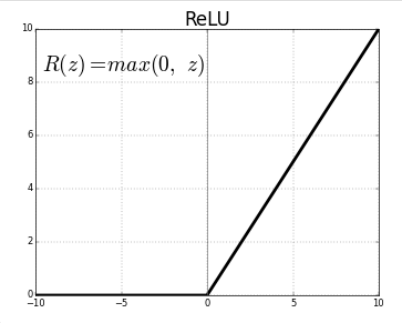


Figure: Source

Universal Approximation Theorem

The reason ANNs are so powerful and they are able to achieve the results that we have seen over the years is mostly due to the fact that ANNs are Universal Approximators:

- Neural networks can approximate any continuous function to an arbitrary degree of accuracy.
- Requires sufficient neurons and layers.
- Activation functions play a key role.

Universal Approximation Theorem

A classical result from Cybenko, in his seminal paper from 1989 is that given a continuous function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, (you will often see this written as $f \in C^0(\mathbb{R}^d)$), for a given accuracy ϵ_n, there exists a number of neurons n, such that:

$$NN(x) = \sum_{i=1}^{n} h(w^T x + b)$$

$$||f - NN||_\infty \leq \epsilon_n$$

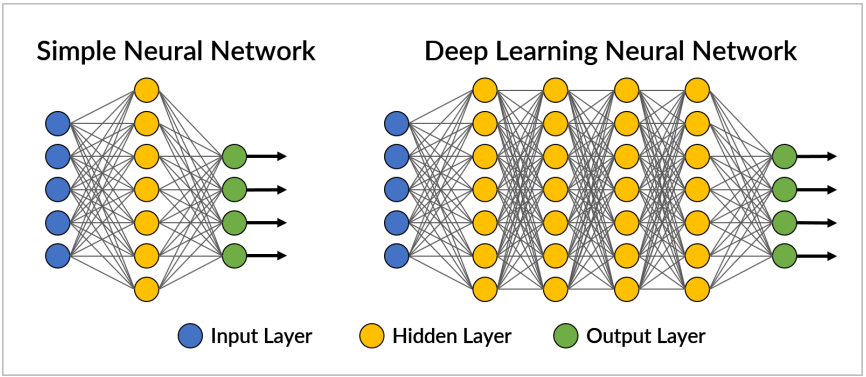
Comments on the multi-layer architecture

- As we have seen, the multi-layer architecture is very powerful.
- However, there are problems. The principle difficulty is developing techniques to find the parameters of the network.
- Another problem is determining, a-priori, now many nodes and layers that the network should have to be able to solve a given problem.

Learning

Recap

We have seen ANNs are very powerful in solving complex pattern recognition (and regression) problems. How do we train them though?



Convention

Each layer has a certain amount of neurons and each neuron will have a certain amount of inputs.

We call W_k and B_k the matrix of weights and the vector of biases for layer k.

$$W_k = \begin{pmatrix} w_{k11} & w_{k12} & \cdots & w_{k1n} \\ w_{k21} & w_{k22} & \cdots & w_{k2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{km1} & w_{km2} & \cdots & w_{kmn} \end{pmatrix}, \quad B_k = \begin{pmatrix} b_{k1} \\ b_{k2} \\ \vdots \\ b_{km} \end{pmatrix}$$

With this convention, calling y_k the output of layer k (with $y_0 = x$), the "forward" operation for each layer is going to be

$$y_k = f_k(W_k y_{k-1} + B_k)$$

Activation Functions

In what follows we will not use the sign(x) function as an activation function. Reason being, sign(x) is non-continuous.

We will consider mostly the sigmoid function:

$$\sigma(\mathsf{x}) = \frac{1}{1 + \mathsf{e}^{-\mathsf{x}}}$$

And the ReLU function

$$\mathsf{ReLU}(\mathsf{x}) = \max(0, \mathsf{x})$$

Even though the ReLU is not differentiable per se, there are a lot of tricks that can be done to fix it (e.g., subgradients).

How do we evaluate performance?

We have two problems now:

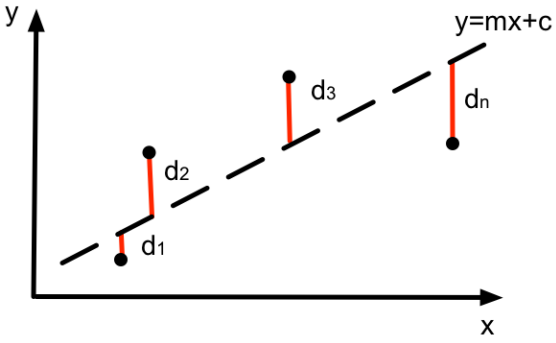
- How do we select appropriate weights?
- How do we evaluate the performance of our network?

These two problems are strongly interconnected to one another. In fact you can show that they can be formalised as:

$$\underset{W}{\textbf{minimize}} \quad J(W)$$

Where $J(W)$ is some appropriate cost function.

Gradient descent for Regression



Consider the following regression problem. We have a bunch of data and we want to find the line such that, the average distance from all of these points to the line is minimal.
In other words, we want to minimise the following cost function

$$J(m, c) = \frac{1}{n}\left(d_1^2 + d_2^2 + ... + d_n^2\right)$$

Gradient descent for Regression

For this specific problem, given n data points, we form the cost function

$$J(m, c) = \frac{1}{n} \sum_{i=1}^{n} (mx(i) + c - y(i))^2$$

We seek to minimize $J(m, c)$ to find the line that best represents the given data.

Gradient descent

Gradient descent starts by calculating the partial derivative of every parameter.

$$\begin{aligned} \frac{\partial \mathsf{J}(\mathsf{m},\mathsf{c})}{\partial \mathsf{m}} &= \frac{1}{\mathsf{n}} \sum_{\mathsf{i}=1}^{\mathsf{n}} -2\mathsf{x}(\mathsf{i})(\mathsf{y}(\mathsf{i}) - (\mathsf{m}\mathsf{x}(\mathsf{i}) + \mathsf{c})) \\ \frac{\partial \mathsf{J}(\mathsf{m},\mathsf{c})}{\partial \mathsf{c}} &= \frac{1}{\mathsf{n}} \sum_{\mathsf{i}=1}^{\mathsf{n}} -2(\mathsf{y}(\mathsf{i}) - (\mathsf{m}\mathsf{x}(\mathsf{i}) + \mathsf{c})) \end{aligned} \tag{6}$$

The vector of $(\frac{\partial \mathsf{J}(\mathsf{m},\mathsf{c})}{\partial \mathsf{m}}, \frac{\partial \mathsf{J}(\mathsf{m},\mathsf{c})}{\partial \mathsf{c}})$ is known as the gradient, and usually denoted $\nabla \mathsf{J}(\theta)$ where θ is the parameter vector (m, c).

Loss functions

The training process involves two steps:

- Selecting a loss function
- Applying gradient descent and finding the parameters that minimize the loss function

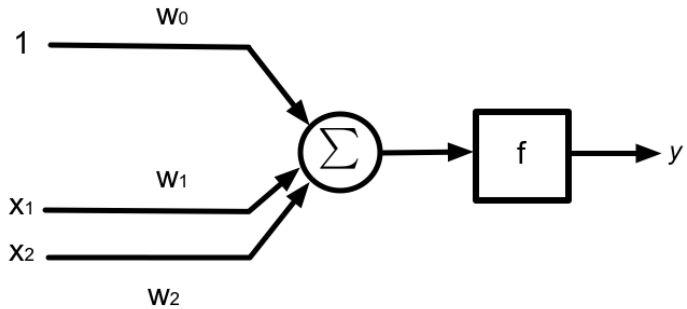
A loss function $J(W)$ is simply a function that indicates how well your model managed to learn the given data.

In the specific case of a neural network, the parameters of the network that we are trying to find, are the weights of each neuron.

Today we are going to use the Least Squared Error (LSE) but we will see at least one more next week.

Single-layer architecture

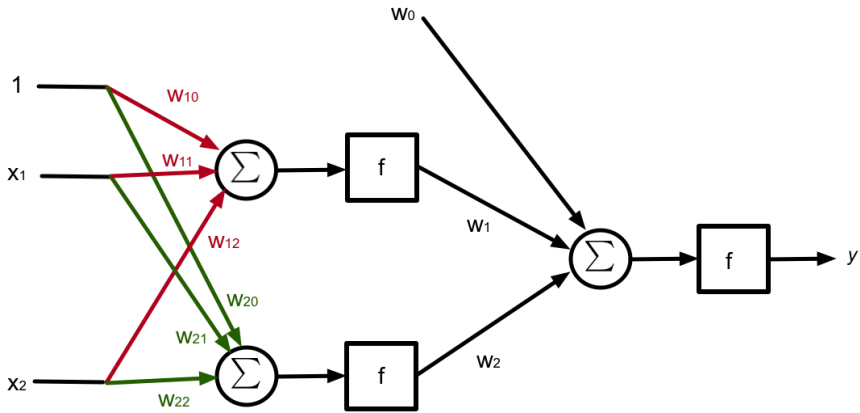
For a simple architecture, gradient descent is fairly straightforward.



$$J((w_0, w_1, w_2)) = \frac{1}{n}\sum_{i=1}^{n}(NN(x(i)) - y(i))^2$$

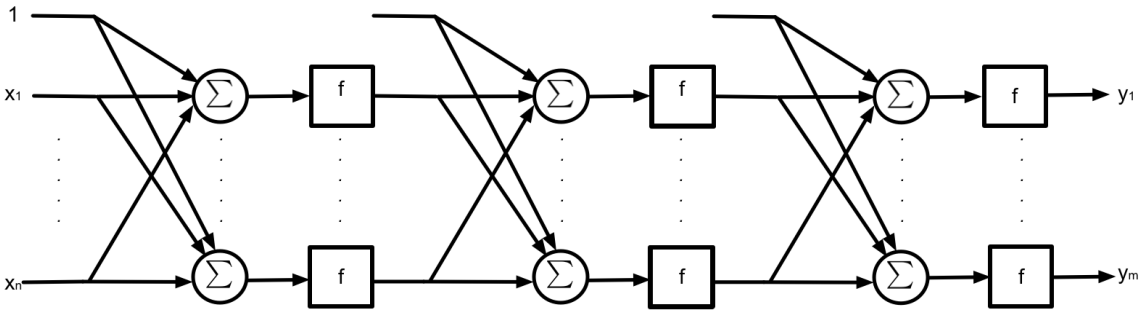
Multi-layer architecture

What about this one?



Multi-layer architecture

And what about this one?



Multi-layer architecture

As in the case of fitting data to a linear model, we wish to use $\nabla J(W)$ to adjust the parameters of the network using the partial derivatives $\frac{\partial J}{\partial w_{ijk}}$:

$$w_{kij}^{m+1} \leftarrow w_{kij}^{m} - \epsilon \frac{\partial J}{\partial w_{kij}} \tag{7}$$

Thus, the main problem is to find a convenient way to calculate the partial derivatives for multi-layer architectures.

Automatic differentiation

A very common technique used in ML is Automatic differentiation.

Basic idea is to use elementary computations and clever algorithms to build up partial derivatives, for example using the chain rule for partial derivatives multi-variate functions.

Back-propagation (AD)

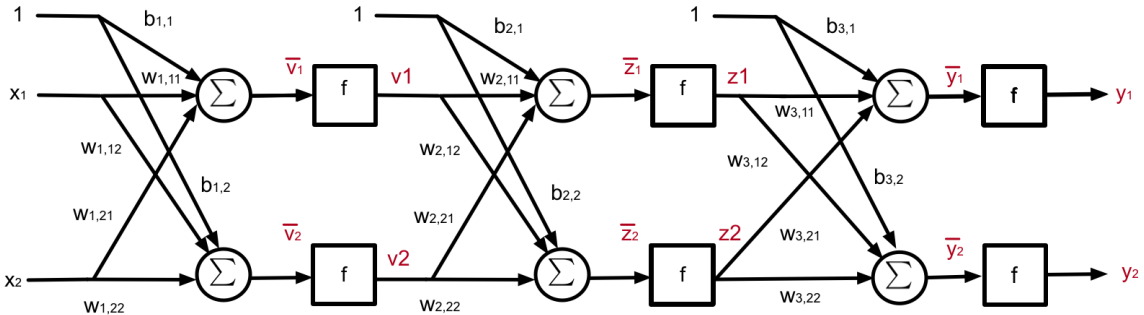
The algorithm for adjusting the weights to implement gradient descent is known as the back propagation algorithm.

In regression problems the objective of the algorithm is to find the parameters of the neural network so that the following quantity is minimized $\| \mathsf{f}(\mathsf{x}) - \mathsf{NN}(\mathsf{x}) \|$.

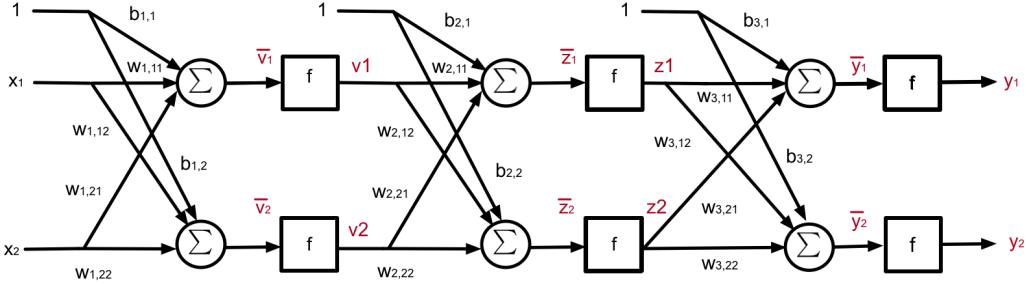
In pattern recognition problems we seek to find a decision boundary $\mathsf{NN}(\mathsf{x}) = 0$, separating several classes, such that the number of misclassifications are minimised.

Multi-layer architecture

To understand the back-propagation algorithm, lets consider one of the simplest multi-layer architecture we can think of. The network has two inputs, and one output. It has 2 hidden layers,2 inputs and 2 output inputs.



Multi-layer architecture



In the above diagram, we have noted the output of each activation function $(\mathsf{y}, \mathsf{z}, \mathsf{v})$ and their inputs $(\bar{\mathsf{y}}, \bar{\mathsf{z}}, \bar{\mathsf{y}})$ in red. For example:

$$\bar{\mathsf{y}}_1 = \mathsf{w}_{3,21}\mathsf{z}_2 + \mathsf{w}_{3,11}\mathsf{z}_1 + \mathsf{b}_{3,1}. \tag{8}$$

Backpropagation

Let us consider the case where data arrives sample by sample. Suppose we have an input $(\mathsf{x}_1(\mathsf{i}), \mathsf{x}_2(\mathsf{i}))$, an output of the neural network $\mathsf{y}_1 = \mathsf{NN}(\mathsf{x}_1(\mathsf{i}), \mathsf{x}_2(\mathsf{i}))$ and some desired output $\mathsf{y}_1(\mathsf{i})$. Let us also define the instantaneous cost function $\mathsf{J}(\theta) = \frac{1}{2}(\mathsf{y}_1 - \mathsf{y}_1(\mathsf{i}))^2 + \frac{1}{2}(\mathsf{y}_2 - \mathsf{y}_2(\mathsf{i}))^2$.

How do we calculate the partial derivatives with respect to all the parameters? Lets start with the partial with respect to $\mathsf{w}_{3,22}$.

$$\frac{\partial \mathsf{J}}{\partial \mathsf{w}_{3,11}} = \frac{\partial \mathsf{J}}{\partial \mathsf{y}_1} \frac{\partial \mathsf{y}_1}{\partial \mathsf{w}_{3,11}} \tag{9}$$

This follows directly from the chain rule.

Backpropagation

Now we can also apply the chain rule again.

$$\frac{\partial \mathsf{J}}{\partial \mathsf{w}_{3,11}} = \frac{\partial \mathsf{J}}{\partial \mathsf{y}_1} \frac{\partial \mathsf{y}_1}{\partial \bar{\mathsf{y}}_1} \frac{\partial \bar{\mathsf{y}}_1}{\partial \mathsf{w}_{3,11}} \tag{10}$$

This follows directly from the chain rule. Everything here can be calculated.

$$\frac{\partial \mathsf{J}}{\partial \mathsf{y}_1} = \mathsf{y}_1 - \mathsf{y}_1(\mathsf{i}); \quad \frac{\partial \mathsf{y}_1}{\partial \bar{\mathsf{y}}} = \mathsf{f}'(\bar{\mathsf{y}}_1); \quad \frac{\partial \bar{\mathsf{y}}}{\partial \mathsf{w}_{3,11}} = \mathsf{z}_1. \tag{11}$$

Backpropagation

Thus it follows that

$$\frac{\partial \mathsf{J}}{\partial \mathsf{w}_{3,11}} = (\mathsf{y}_1 - \mathsf{y}_1(\mathsf{i}))\mathsf{f}'(\bar{\mathsf{y}}_1)\mathsf{z}_1 \tag{12}$$

And

$$\frac{\partial \mathsf{J}}{\partial \mathsf{b}_{3,1}} = (\mathsf{y}_1 - \mathsf{y}_1(\mathsf{i}))\mathsf{f}'(\bar{\mathsf{y}}_1) \tag{13}$$

Backpropagation

What about weights in the second hidden layer?

Calculating the partial derivatives for the second layer is very similar to what we did before. **Caution :** we now need to be very careful with notation.

More specifically, we just need to be careful in using the chain rule for partial derivatives.

I find it really helps to look at the network diagram when doing this step.

Backpropagation

So as before (lets pick $w_{2,11}$) for our calculation:

$$\frac{\partial J}{\partial w_{2,11}} = \frac{\partial J}{\partial \bar{z}_1}\frac{\partial \bar{z}_1}{\partial w_{2,11}} \tag{14}$$

$$= \frac{\partial J}{\partial z_1}\frac{\partial z_1}{\partial \bar{z}_1}\frac{\partial \bar{z}_1}{\partial w_{2,11}} \tag{15}$$

$$= \frac{\partial J}{\partial z_1} f'(\bar{z}_1) v_1 \tag{16}$$

$$= \left(\frac{\partial J}{\partial \bar{y}_1}\frac{\partial \bar{y}_1}{\partial z_1} + \frac{\partial J}{\partial \bar{y}_2}\frac{\partial \bar{y}_2}{\partial z_1}\right) f'(\bar{z}_1) v_1 \tag{17}$$

(18)

Backpropagation

As before everything is known

$$
\begin{aligned}
&= \left(\frac{\partial J}{\partial \bar{y}_1}\frac{\partial \bar{y}_1}{\partial z_1} + \frac{\partial J}{\partial \bar{y}_2}\frac{\partial \bar{y}_2}{\partial z_1}\right) f'(\bar{z}_1) v_1 \\
&= \left((y_1 - y_1(i)) f'(\bar{y}_1) w_{3,11} + (y_2 - y_2(i)) f'(\bar{y}_2) w_{3,12}\right) f'(\bar{z}_1) v_1
\end{aligned}
\tag{19}
$$

Backpropagation

All other parameters in the second layer can be calculated in this manner.

What about the first layer?

Backpropagation

Since all remain partial derivatives have already been calculated in the calculations in layer 2, we are now in a position to update the weights of the network according to:

$$w_{kij} \leftarrow w_{kij} - \alpha \frac{\partial J}{\partial w_{kij}} \tag{20}$$

Since we update the gradient every time we see a new sample, we can do this step as we process each sample.

Or we can calculate the gradient for each parameter for every sample, and average these values over the entire batch. This is called batch gradient descent, and corresponds to what we did before.

Thanks for your attention! Any questions?